

AMONG US

Code Architecture Design Document

Multiplayer Online Game — Software Construction & Development

Game Programming Architecture Assignment

BSSE — Spring 2025

Game Selected	Among Us
Genre	Social Deduction / Multiplayer
Players	4 – 15 Players per Match
Architecture	Client-Server with Authoritative Host
Language Ref.	C# (Unity Engine)

1. Game Overview

Among Us is an online multiplayer social deduction game developed and published by InnerSloth. It supports 4 to 15 players in a space-themed setting where most players are Crewmates trying to complete tasks and identify Impostors, while a smaller group of Impostors secretly sabotage the crew and eliminate Crewmates without being detected.

The game gained massive global popularity in 2020 and is a strong example of a lightweight real-time multiplayer system with discrete game phases, voting mechanics, and emergent social interaction — making it an excellent subject for architecture analysis.

1.1 Core Gameplay Mechanics

Task System	Crewmates complete short mini-game tasks scattered across the map.
Sabotage System	Impostors trigger map-wide events (reactor meltdown, O2 failure) to cause chaos.
Kill System	Impostors eliminate Crewmates with a cooldown-based kill action.
Reporting	Any player can report a dead body, triggering an Emergency Meeting.
Voting System	All living players vote to eject a suspected Impostor. Majority wins.
Win Conditions	Crew wins by completing all tasks OR ejecting all Impostors. Impostors win by outnumbering Crew or completing a sabotage timer.
Ghost Role	Dead players become Ghosts — they can still complete tasks but cannot interact.
Vent System	Impostors can travel instantly between vent shafts hidden across the map.

2. High-Level Architecture

Among Us uses a Host-Client architecture where one player's machine acts as the authoritative host (server) and all other players are clients. The host validates game state changes, resolves conflicts, and broadcasts updates to all clients. Below is the top-level system diagram.

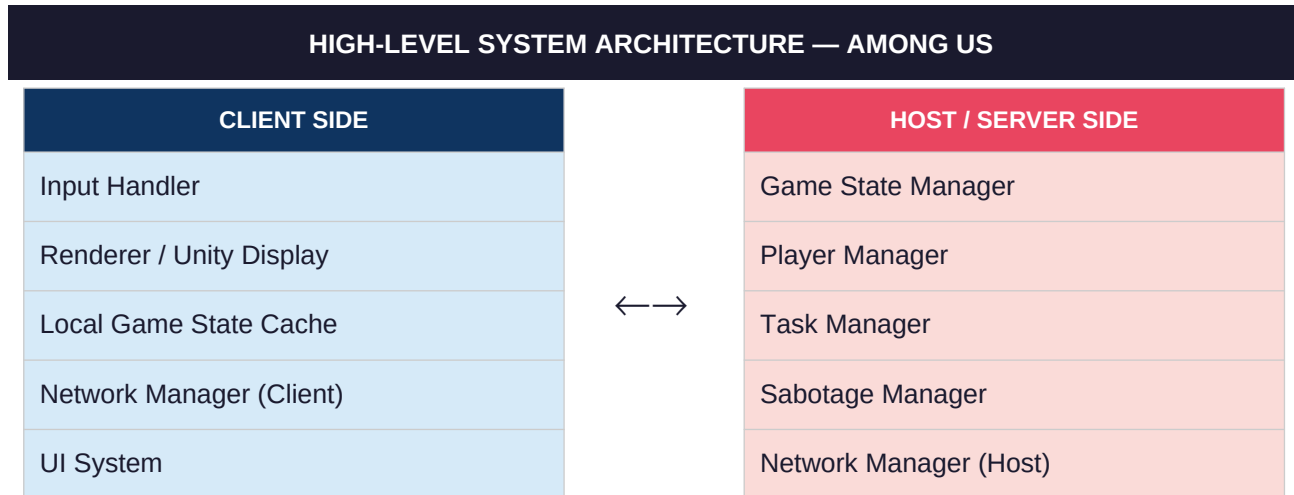


Figure 1: Client-Host communication model

2.1 Major Systems

Player System	Manages player identity, role assignment (Crewmate/Impostor), movement, animation state, color, and connection lifecycle. Each player is represented by a PlayerController object synced across all clients.
Matchmaking System	Handles lobby creation, join codes, region selection, and player slot management. Uses Photon or a custom relay server to connect players before the game starts.
Game State Manager	Central FSM (Finite State Machine) controlling game phases: Lobby → In-Game → Discussion → Voting → Results → End. All systems subscribe to phase transitions.
Task System	Stores a global task list. Each Crewmate is assigned a subset. Tasks have individual completion states. The TaskBar (global progress bar) is updated by the host when any task is completed.
Combat/Kill System	Impostor-only system. Tracks kill cooldown timer, validates kill range, removes the target player from the living list, and spawns a dead body GameObject at kill location.

Sabotage System	Impostors trigger sabotages with cooldowns. Each sabotage type (lights, reactor, O2, comms) has its own handler that modifies global game state and starts a countdown. Crew must respond within time or Impostors win.
Voting System	Activated on body report or emergency button. Collects votes from all living players within a time window, tallies results on host, and executes ejection if majority agrees.
UI System	Renders HUD elements: task list, kill button, sabotage menu, chat, vote UI, meeting overlay, and results screen. Driven by game state events.
Network Manager	Handles serialization of game events (RPCs), player position sync, lag compensation, and reconnection logic. Uses UDP for real-time position updates and TCP-like reliability for critical events (kills, votes).

3. Code Structure

The codebase is organized using Unity's MonoBehaviour component system combined with a Manager-Controller-Model pattern. Singletons are used for global managers, while individual entities (players, tasks, bodies) use component-based design.

3.1 Folder Structure

```
Assets/
Scripts/
Core/ – GameManager, GameStateMachine, NetworkManager
Player/ – PlayerController, PlayerMovement, PlayerRole, GhostController
Systems/
Tasks/ – TaskManager, BaseTask, ElectricalTask, MedBayTask ...
Sabotage/ – SabotageManager, ReactorHandler, O2Handler, LightsHandler
Voting/ – VotingManager, VoteUI, MeetingController
Kill/ – KillManager, DeadBody, KillButton
Vent/ – VentManager, Vent, VentPath
UI/ – HUDManager, TaskListUI, ChatManager, LobbyUI
Matchmaking/ – LobbyManager, RegionSelector, JoinCodeHandler
Network/ – RPCHandler, StateSyncer, EventBus
Utils/ – Constants, Extensions, EnumTypes
Prefabs/ – Player, DeadBody, Task objects, UI Panels
Scenes/ – MainMenu, Lobby, Skeld, Mira HQ, Polus, Airship
```

3.2 Core Class Definitions

class GameManager		: Singleton
GamePhase currentPhase		
List allPlayers		
void ChangePhase(GamePhase newPhase)		
void CheckWinCondition()		
void OnPlayerDeath(PlayerController p)		

class PlayerController		: MonoBehaviour
string playerName		
PlayerRole role // CREWMATE IMPOSTOR		
bool isAlive		
bool isHost		
void Move(Vector2 direction)		

```
void Kill(PlayerController target)
```

```
void CompleteTask(BaseTask task)
```

```
void ReportBody(DeadBody body)
```

```
class TaskManager : Singleton
```

```
List allTasks
```

```
float globalTaskProgress // 0.0 to 1.0
```

```
void AssignTasksToPlayer(PlayerController p)
```

```
void OnTaskCompleted(BaseTask task)
```

```
bool AreAllTasksDone()
```

```
class VotingManager : Singleton
```

```
Dictionary votes // voter -> target
```

```
float votingTimeRemaining
```

```
void StartMeeting(string reporterId)
```

```
void CastVote(string voterId, string targetId)
```

```
void TallyAndEject()
```

```
class SabotageManager : Singleton
```

```
SabotageType activeSabotage
```

```
float sabotageTimer
```

```
float sabotageCooldown
```

```
void TriggerSabotage(SabotageType type)
```

```
void ResolveSabotage()
```

```
void OnSabotageTimeout()
```

```
class NetworkManager : Singleton
```

```
bool isHost
```

```
List connections
```

```
void SendRPC(string methodName, object[] args)
```

```
void BroadcastState(GameStateSnapshot snap)
```

```
void OnClientDisconnect(string clientId)
```

3.3 Game State Machine (FSM)

The entire game flow is governed by a Finite State Machine with the following states and transitions:

STATE MACHINE DIAGRAM		
LOBBY	Players join, set options	↓
IN-GAME	Tasks, kills, sabotage active	↓
DISCUSSION	Body reported / emergency button	↓
VOTING	Players cast votes, timer	↓
EJECT	Result shown, player ejected	↓
GAME OVER	Win/loss screen displayed	

Figure 2: Among Us Game Phase FSM

3.4 Communication Between Systems

Systems communicate through an Event Bus (Observer Pattern). No system directly calls another — instead they publish and subscribe to typed events. This keeps systems decoupled and testable.

Publisher	Event	Subscribers
PlayerController	OnPlayerDied	GameManager, VotingManager, UIManager
TaskManager	OnAllTasksComplete	GameManager
SabotageManager	OnSabotageTriggered	UIManager, LightsHandler, TaskManager
SabotageManager	OnSabotageTimeout	GameManager (Impostor Win)
VotingManager	OnPlayerEjected	GameManager, UIManager, PlayerManager
GameManager	OnPhaseChanged(GamePhase)	All systems (subscribe individually)
NetworkManager	OnClientDisconnected	GameManager, PlayerManager

Table 1: Event Bus Communication Map

3.5 Network Architecture & RPC System

Among Us uses a Host-Authoritative model. The host processes all game-critical events (kills, votes, sabotages). Clients send input requests; host validates and broadcasts results. Two communication channels are used:

Channel	Protocol	Used For	Priority
State Sync	UDP	Player positions, animations	High Frequency, Low Priority
Game Events	Reliable	Kills, votes, phase changes, tasks	Low Frequency, High Priority
Chat	TCP-like	Text messages during meetings	Low Frequency, Medium Priority
Lobby Events	Reliable	Player join/leave, settings change	Low Frequency, High Priority

Table 2: Network Channels

A typical kill event flow looks like this:

Step	Actor	Action
1	Impostor Client	Press Kill button → Send KillRequest RPC to Host
2	Host	Validate: range check, cooldown check, alive check
3	Host	If valid: mark player dead, broadcast KillResult RPC
4	All Clients	Receive KillResult → remove player, spawn DeadBody
5	Host	Check win condition (enough Impostors?)

Figure 3: Kill Event RPC Flow

4. Design Decisions & Justification

4.1 Host-Authoritative Model

Why chosen?

Dedicated servers are expensive for an indie game with up to 15 players. Using one player's machine as host eliminates server costs while keeping game logic centralised. The host validates all critical actions (kills, votes) so no client can cheat by sending false RPCs directly. This is practical for social deduction games where real-time physics precision is not required.

4.2 Finite State Machine for Game Flow

Why chosen?

Among Us has very distinct, non-overlapping game phases. FSM makes transitions explicit, predictable, and auditable. Systems simply subscribe to phase change events and enable/disable themselves accordingly. This avoids spaghetti conditional logic spread across multiple classes and makes adding new phases (e.g., a new game mode) trivial.

4.3 Event Bus (Observer Pattern) for Inter-System Communication

Why chosen?

Direct references between managers create tight coupling and circular dependencies. An Event Bus lets TaskManager announce 'tasks complete' without knowing GameManager exists. This enables unit testing each system in isolation and allows future systems (e.g., analytics, replay recorder) to subscribe without modifying existing code.

4.4 Component-Based Player Architecture

Why chosen?

Separating PlayerMovement, PlayerRole, PlayerAnimator into distinct components follows Unity best practices and the Single Responsibility Principle. Impostors and Crewmates share the same PlayerController base but activate different component sets via their role assignment. Ghost players simply disable the physical components and enable GhostController, reusing the same prefab.

4.5 Dual-Channel Networking (UDP + Reliable)

Why chosen?

Player position updates are sent 20 times/second and dropped packets are acceptable — old position data is useless anyway. UDP minimizes latency for movement. However, game events (kills, votes, task completions) must never be lost, so a reliable channel with acknowledgment and retransmission is used. This hybrid approach balances responsiveness with correctness.

4.6 Singleton Pattern for Global Managers

Why chosen?

Managers like GameManager, TaskManager, and NetworkManager are accessed frequently from many systems and only one instance should ever exist per game session. Singletons provide a globally accessible, single-instance reference. While overuse of singletons is an anti-pattern, for game managers that are truly global (not per-scene), they are the most pragmatic and readable solution in Unity.

5. Architecture Summary

The table below provides a concise summary of every architectural decision, the pattern applied, and its primary benefit:

Concern	Pattern / Approach	Benefit
Game Flow Control	Finite State Machine	Clear phase transitions, easy to extend
Inter-System Comms	Event Bus / Observer	Loose coupling, testable systems
Player Entity	Component-Based (Unity)	Reusable, role-switchable prefabs
Global Managers	Singleton	Single instance, global access
Network Authority	Host-Authoritative	No dedicated server cost, cheat prevention
Position Sync	UDP State Sync (20Hz)	Low latency movement
Game Events Sync	Reliable RPC	No critical event loss
Win Condition Check	Observer on State Change	Automatic, decoupled evaluation
Role Behaviour	Strategy Pattern	Crewmate/Impostor swap without duplication
Task Assignment	Factory + Random Sampling	Fair, configurable task distribution
Sabotage Handling	Command Pattern	Undoable, extendable sabotage actions